AgentPerf: Measuring TTFT and Approximating TPOT at the Model-Provider Boundary

April 18, 2026

Abstract

This paper describes the design and validation of `agentperf`, a local OpenAI-compatible gateway built to measure latency and streaming behavior at the boundary between an OpenAI client and a configured model provider. The work began with a narrower question: whether Codex transcripts, together with the Responses API's content-delta stream, were sufficient to estimate time to first token and time per output token without modifying the client. That path turned out to be inadequate. Codex transcripts preserve high-level request and tool activity, but they do not consistently preserve the low-level streaming records, transport timing, or raw event cadence needed to support defensible latency analysis. The result was a shift in architecture from passive transcript analysis to an active local gateway that sits in the `base_url` position for any OpenAI-compatible client.

The gateway now forwards real traffic to any configured provider profile, captures server-sent event timing, stores request and stream metadata in SQLite, runs provider-scoped latency probes, and produces comparison reports across providers such as a direct OpenAI endpoint and an enterprise gateway such as `amd_gateway`. More importantly, live use against those endpoints changed the implementation. The original hope was that TPOT could be approximated directly from post-first-token stream time and reported output-token counts. In practice, actual streaming behavior was often much more complicated. Some endpoints delivered long sequences of small text deltas over several seconds, while others withheld almost all output and then flushed the result in a handful of dense bursts. That observation led to a more cautious metric model, including burst-structure analysis and a cadence-based TPOT recommendation for heavily buffered streams.

1. The Original Problem

The starting point for this work was not "build a proxy," but rather "measure TTFT and TPOT as economically as possible." The initial hypothesis was that the Responses API already exposes enough information through content-delta streaming to approximate those values from ordinary Codex runs. If Codex transcripts under `~/.codex` already preserved the original prompt, the streamed text deltas, and reliable timestamps, then a separate measurement layer would have been unnecessary.

That hypothesis failed for two reasons. First, Codex transcripts do not function as a transport-level packet capture. They preserve a useful operational trace, but not the full sequence of low-level upstream response events needed for consistent timing analysis. Second, the available client-side extension points did not provide a clean way to observe raw upstream Responses API traffic without either patching the client or relying on provider-specific internals. Once those limitations became clear, the design requirement changed. The measurement system had to be external to the client, transparent to the client, and positioned on the network path where it could see the actual request and response stream.

That conclusion is what produced the gateway architecture. Instead of trying to infer timing from partial transcripts after the fact, `agentperf` acts as the local `base_url` target for an OpenAI-compatible client. The client sends normal traffic to the gateway. The gateway forwards it to the true provider, returns the provider's response unchanged, and retains just enough timing structure to make the latency analysis reproducible.

2. Why a Local Gateway Was the Right Abstraction

The choice of a gateway was not merely convenient; it aligned the measurement boundary with the real question. In environments that use enterprise model gateways, the user often does not control the ultimate model-serving infrastructure. A request may flow through a corporate gateway and only then reach a public model API or some further internal service. At that point, "provider latency" becomes ambiguous. The gateway implemented here resolves that ambiguity by measuring at the boundary of the provider profile the client is configured to call. If the configured provider is `amd_gateway`, then the measurements describe the `amd_gateway` boundary. If the configured provider is direct OpenAI, they describe the OpenAI boundary. The system does not pretend to see through an intermediary it is not actually connected to.

That same abstraction also made the implementation fit naturally with Codex. Codex already has the notion of model providers, including provider-specific `base_url`, static HTTP headers, and headers expanded from environment variables. The proxy therefore adopted Codex-style provider configuration and can import `[model_providers.*]` blocks directly from `~/.codex/config.toml`. This meant that an existing Codex environment using `amd_gateway` could be re-pointed at the local gateway with minimal friction, while a direct OpenAI provider could be added locally for comparison work. In effect, the proxy became a measurement-preserving relay that looks like an OpenAI endpoint to the client and like an ordinary client to the upstream provider.

3. Implementation Overview

The implemented system exposes an OpenAI-compatible HTTP surface for `POST /v1/responses` and `GET /v1/responses/{response_id}`. For streaming requests, it parses server-sent events as they arrive and records the timestamps of `response.created`, the first text delta, each text-delta event, and the terminal response event. Request metadata and stream events are persisted to SQLite through a queue-backed writer so that file I/O does not dominate the request path. A background probe worker measures provider-scoped round-trip latency, and the benchmark harness can also request explicit probes for non-active providers before a measured batch begins.

The most important design choice here is that the system retains monotonic timing at the point where bytes are actually observed. TTFT is not inferred from wall-clock log messages, and TPOT is not inferred from narrative transcript structure. The proxy timestamps the upstream request dispatch, the stream-open point, the first text delta, the last text-delta receive instant, and the terminal response event. That choice is what made later revisions possible. When live data contradicted the first TPOT approximation, the underlying event history was still rich enough to support a better one.

The system also had to support real multi-client use. That is why upstream HTTP clients are pooled, why SQLite writes are buffered behind a queue-backed worker, and why provider-scoped probe samples are cached in memory for quick association with individual request rows. In practice, this means the gateway is not only an experimental instrument; it is usable as a local service for repeated benchmarking and comparative runs.

4. Metric Model and Its Revision

The initial TTFT and TPOT model was intentionally simple. TTFT raw was defined as the time from client request ingestion to the first observed text delta. For a stream that delivers output incrementally, TPOT was defined as post-first-text stream time divided by the number of output tokens after the first token. That formulation is still reasonable for a genuinely streaming endpoint. It is easy to explain, easy to compute, and directly tied to what the client saw.

The problem is that "what the client saw" is not always equivalent to "how the model decoded." Live traffic made that distinction impossible to ignore. Some streams looked like ordinary incremental delivery. Others looked like a long silent period followed by an extremely dense burst of deltas, many arriving at the exact same monotonic timestamp. In that pattern, dividing a tiny post-first-text interval by the full token count yields a mathematically correct number that is physically misleading. It measures flush behavior, not decode cadence.

The current metric model therefore keeps the original TPOT fields, but no longer treats them as the default interpretation in all cases. The benchmark harness now derives burst-structure metrics from the stored stream events, including the number of distinct receive instants that contained text deltas, the mean number of delta events per receive instant, the maximum burst size, and the span from the first text-delta receive instant to the last one. Those derived metrics allow the system to classify a sample as `streaming`, `mixed`, or `bursty`.

For bursty samples, the harness now recommends a different TPOT approximation. Instead of looking only at the time after the first text delta, it measures an output-cadence window from upstream request dispatch to the last observed text delta, subtracts the provider probe RTT as a coarse transport adjustment, and then divides by total output tokens. This does not claim to reveal the provider's internal token loop. It does, however, avoid the absurd outcome in which an endpoint appears to decode at ten thousand tokens per second simply because it chose to flush buffered content late and quickly.

5. Short-Response Results

The short-prompt benchmark used a modest response budget and a prompt designed to elicit roughly one hundred words of output. In that run, `amd_gateway` reached first text far sooner than direct OpenAI. The median raw TTFT was about 1.32 seconds through `amd_gateway` and about 4.37 seconds through direct OpenAI. Full latency was closer, but still favored `amd_gateway` by roughly half a second. If one stopped there, the conclusion would simply be that `amd_gateway` produced a materially faster first token on that host and workload.

The TPOT numbers from the same run told a stranger story. `amd_gateway` showed a median post-first-text TPOT of about 18.6 milliseconds per token, which is entirely plausible for an observed delivery cadence. Direct OpenAI showed a median of about 0.23 milliseconds per token, which is not plausible as a token-by-token decode rate for a model generating natural-language output. The data were not wrong; the interpretation was. What the numbers really said was that direct OpenAI, in this environment, waited longer for first text and then delivered the response in a very tight burst.

6. Long-Response Results and the Emergence of Burst Analysis

Because short prompts can mask or exaggerate streaming behavior, a second benchmark used a much longer prompt and a much larger output budget. That run is the clearest expression of the traffic patterns this project has uncovered. Through `amd_gateway`, the representative sample produced first text in roughly 1.23 seconds and then continued to deliver text over roughly 18.7 seconds. Through direct OpenAI, the representative sample produced first text only after roughly 19.6 seconds, then delivered the remaining text over about 86 milliseconds.

Figure 1 shows the request timeline for representative long-prompt samples from each endpoint.

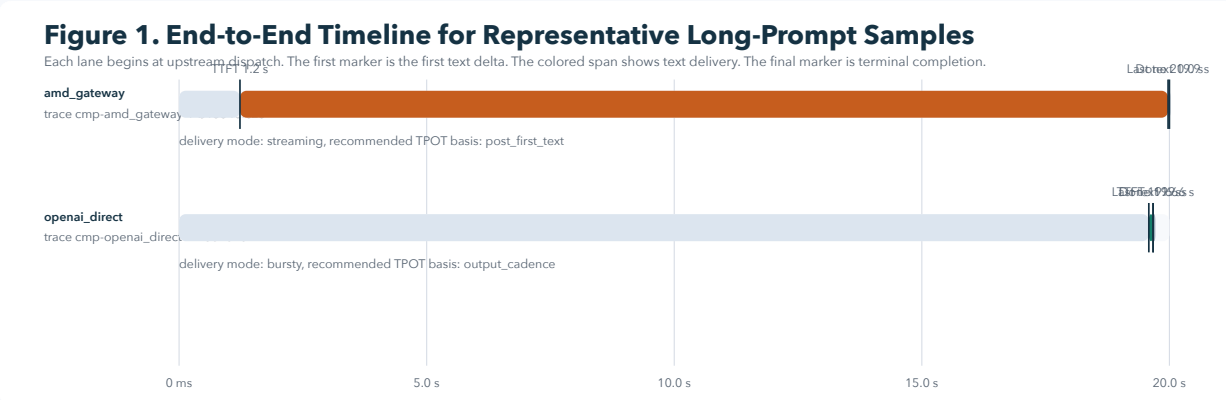

Figure 1: Response timeline

Figure 1 is the most compact statement of the problem the proxy was built to solve. The `amd_gateway` path begins producing text quickly and continues to stream it. The direct OpenAI path holds the response much longer before releasing it. If one only compares full latency, the two endpoints look surprisingly similar for the long prompt. If one only compares TTFT, `amd_gateway` looks dramatically better. If one only compares post-first-text TPOT, direct OpenAI looks impossibly fast. All three facts are true at once because they describe different aspects of the same traffic pattern.

Figure 2 shows the receive-side burst structure after first text.

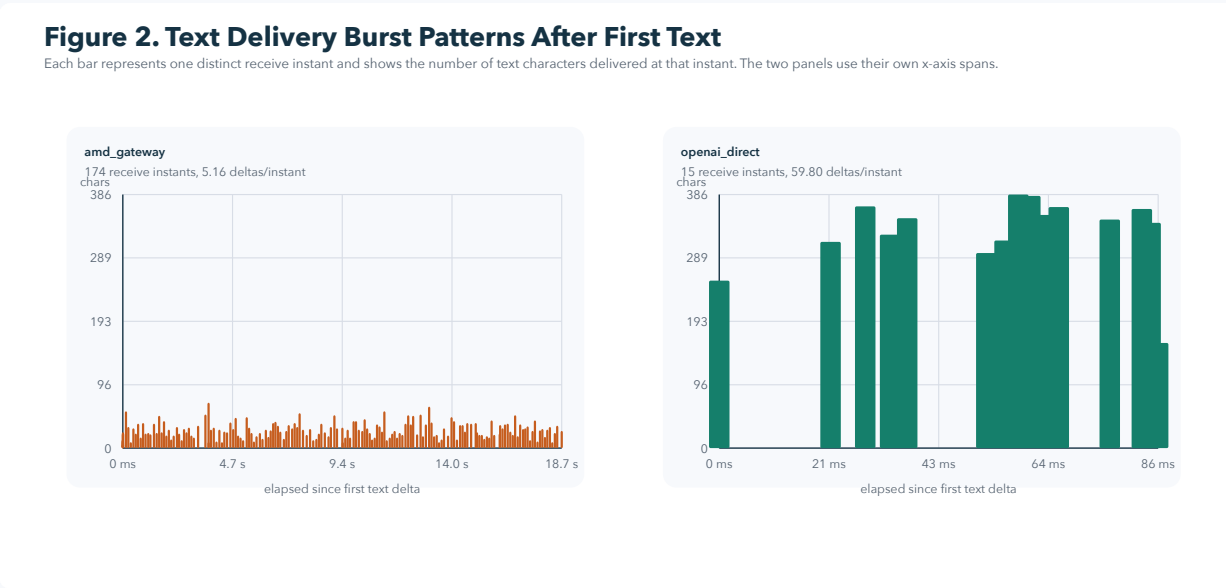

Figure 2: Burst patterns

The contrast is stark. In the representative long-prompt sample, `amd_gateway` spread 897 text-delta events across 174 distinct receive instants. Direct OpenAI delivered the same 897 text-delta events across only 15 receive instants, with individual bursts containing around sixty delta events and several hundred characters. That is not a small methodological wrinkle. It is exactly the kind of behavior that invalidates naive TPOT inference from post-first-text stream time.

7. Comparison View

Figure 3 summarizes the comparison across short and long workloads.

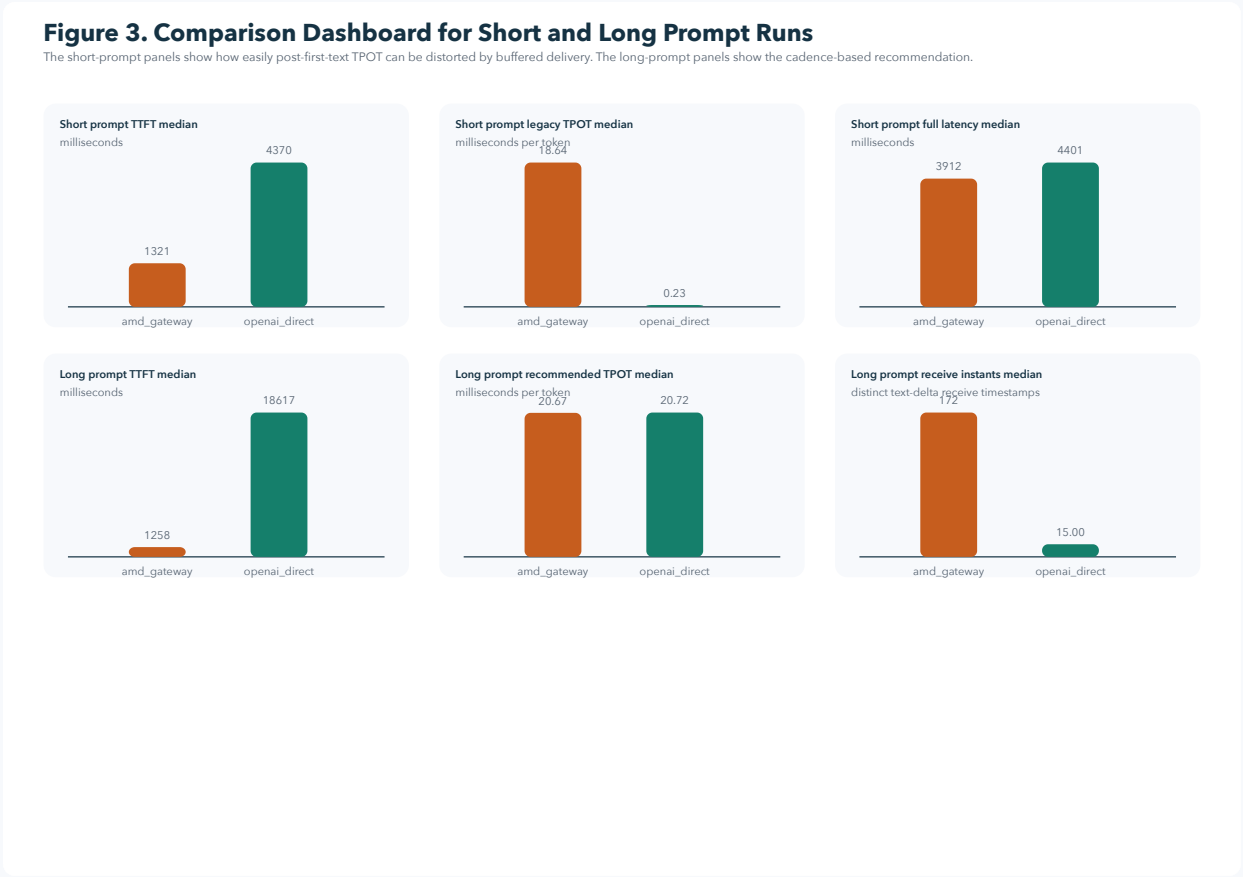

Figure 3: Comparison dashboard

The dashboard makes two points that are easy to miss in prose alone. First, short-prompt TPOT is fragile. The short run makes direct OpenAI appear thousands of times faster than `amd_gateway` on a tokens-per-second basis, even though the long-prompt run shows that most of that "advantage" is really buffering and flush behavior. Second, the cadence-based

recommendation changes the comparison materially. On the long prompt, the old post-first-text TPOT for direct OpenAI was about 0.10 milliseconds per token, while the cadence-based recommendation brought it back to about 20.7 milliseconds per token. That result is close to `amd_gateway`'s own recommended TPOT of about 20.7 milliseconds per token on the same workload.

This does not prove that the two providers decode identically. It does show that once buffering behavior is accounted for, the huge apparent TPOT gap largely disappears. The remaining dominant difference is TTFT: one provider starts talking much earlier, while the other withholds output and then emits it rapidly.

8. How Live Use Changed the Implementation

Several parts of the current implementation exist only because live measurements forced them into existence. Provider-scoped explicit probes were added because a single continuously probed active provider was not enough for reliable side-by-side comparisons. `response.incomplete` had to be treated as terminal because some real streams ended that way while still carrying valid usage information. The adjusted TTFT model had to move away from a half-RTT subtraction based on a later header boundary and instead use dispatch-to-first-text minus probe RTT, because the earlier version was too brittle to defend.

Most significantly, TPOT reporting had to become two-tiered. The original post-first-text TPOT is still valuable for truly incremental streams, and discarding it would throw away useful information. At the same time, live direct-OpenAI traffic demonstrated that an endpoint can be perfectly healthy and still make that metric nonsensical. The current implementation therefore preserves the raw metric, derives burst structure, and emits a recommended TPOT basis that changes according to the observed delivery mode. That is a direct consequence of what the proxy saw in the field, not an abstract refinement added in isolation.

9. Limits of the Current System

The gateway measures at the configured provider boundary, not inside the provider. It knows when bytes were dispatched and when bytes were observed, but it does not know when the model itself completed any particular token internally. The RTT adjustment is therefore only an approximation, and even the cadence-based TPOT should be read as a coarse boundary estimate rather than a provider-internal truth.

The current delivery-mode classification is also heuristic. It is grounded in real traffic and is already useful, but it is not yet a formal statistical model. More validation across providers, models, and network paths would improve confidence. The current charts and summaries come from relatively small sample counts collected on a single host on April 18, 2026. The implementation is now capable of larger campaigns; the present paper should be read as a description of the method and an explanation of what the method revealed in early live use.

10. Conclusion

The central contribution of this work is not merely a proxy, but a better measurement posture. The project began with the hope that transcript inspection might be enough to recover TTFT and TPOT from ordinary client behavior. It ended with a local gateway because the low-level timing evidence needed for defensible analysis has to be captured where the traffic is actually observed. That architectural change made it possible to discover an even more important fact: TPOT is not a single universally trustworthy number when providers differ in how they flush streamed content.

`agentperf` now makes those differences explicit. It can compare direct and gateway-mediated endpoints through the same client-facing interface, preserve request and stream structure, attach provider-scoped network probes, reveal burstiness in the event stream, and replace a misleading TPOT with a more defensible cadence-based recommendation when the traffic pattern demands it. That is a meaningful step toward practical, provider-boundary latency analysis for OpenAI-compatible systems.

The figures in this paper were generated from the measured snapshot in whitepaper_assets/live_data_snapshot.json, derived from the April 18, 2026 benchmark reports captured through the proxy.